

GIẢI QUYẾT VẤN ĐỀ VÀ TÌM KIẾM

CHƯƠNG 3, PHẦN 1–5

Nội dung

- ◇ Tác nhân giải quyết vấn đề
- ◇ Các loại bài toán
- ◇ Khởi tạo bài toán
- ◇ Các bài toán ví dụ
- ◇ Các thuật toán tìm kiếm cơ bản

Tác nhân giải quyết vấn đề

Một dạng hạn chế của tác nhân tổng quát:

```
function SIMPLE-PROBLEM-SOLVING-AGENT(p) returns an action
inputs: p, a percept
static: s, an action sequence, initially empty
         state, some description of the current world state
         g, a goal, initially null
         problem, a problem formulation

state ← UPDATE-STATE(state, p)
if s is empty then
  g ← FORMULATE-GOAL(state)
  problem ← FORMULATE-PROBLEM(state, g)
  s ← SEARCH(problem)
action ← RECOMMENDATION(s, state)
s ← REMAINDER(s, state)
return action
```

Lưu ý: đây là giải quyết vấn đề *ngoại tuyến (offline)*.

Giải quyết vấn đề *trực tuyến (online)* bao gồm việc hành động mà không có tri thức đầy đủ về bài toán và giải pháp.

Ví dụ: Romania

Đang đi nghỉ ở Romania; hiện đang ở Arad.

Chuyến bay khởi hành vào ngày mai từ Bucharest

Định dạng mục tiêu:

có mặt ở Bucharest

Khởi tạo bài toán:

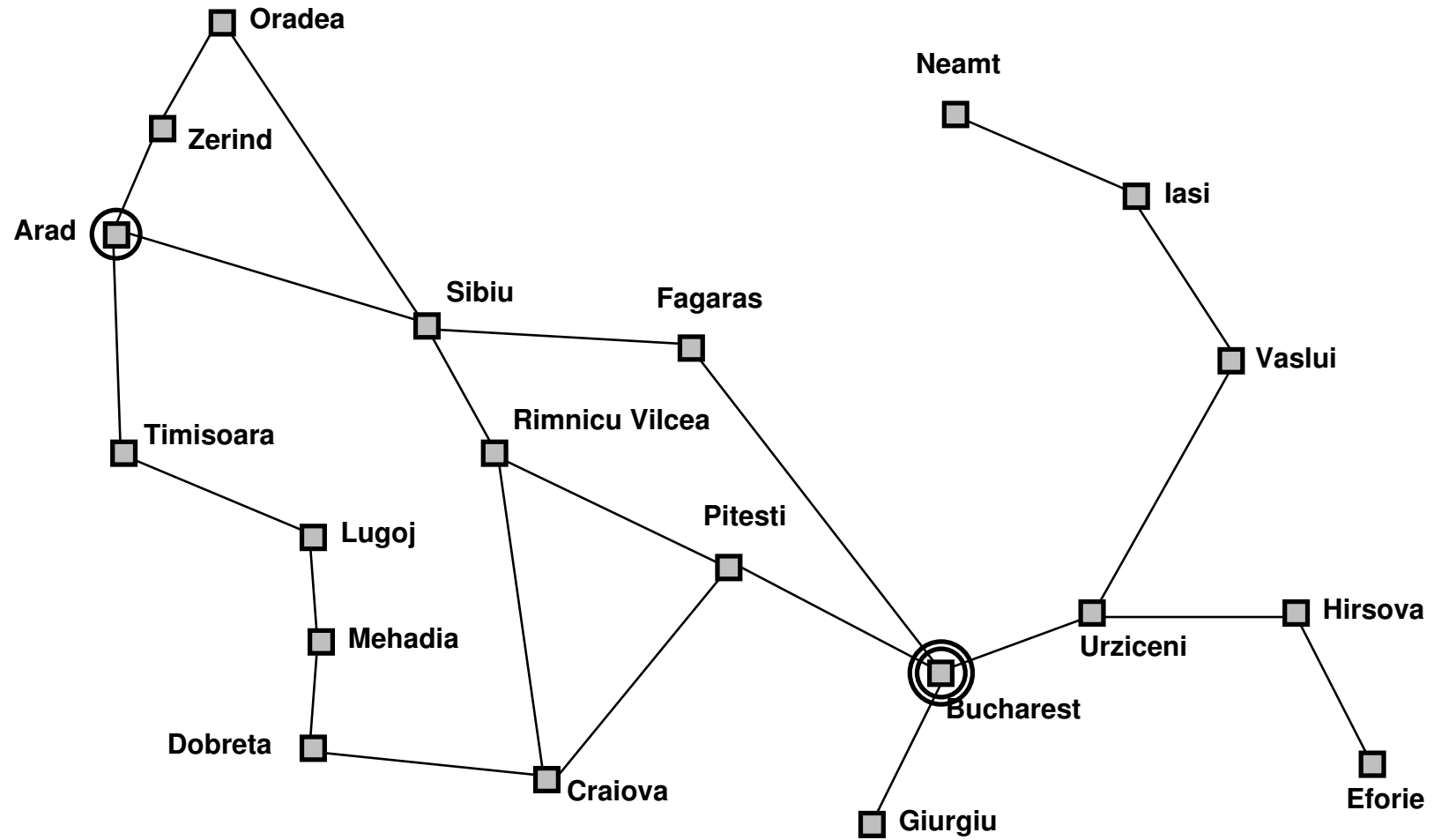
trạng thái: các thành phố khác nhau

toán tử: lái xe giữa các thành phố

Tìm giải pháp:

chuỗi các thành phố, ví dụ: Arad, Sibiu, Fagaras, Bucharest

Ví dụ: Romania



Các loại bài toán

Tất định, có thể truy cập $\implies$ bài toán trạng thái đơn (*single-state*)

Tất định, không thể truy cập $\implies$ bài toán đa trạng thái
(*multiple-state*)

Không tất định, không thể truy cập $\implies$ bài toán dự phòng
(*contingency*)

phải sử dụng cảm biến trong quá trình thực thi

giải pháp là một *cây* hoặc *chính sách*

thường *đan xen* giữa tìm kiếm và thực thi

Không gian trạng thái chưa biết $\implies$ bài toán khám phá (“trực tuyến”)

Ví dụ: thể giới máy hút bụi

Trạng thái đơn, bắt đầu ở #5. ˘Giải pháp??

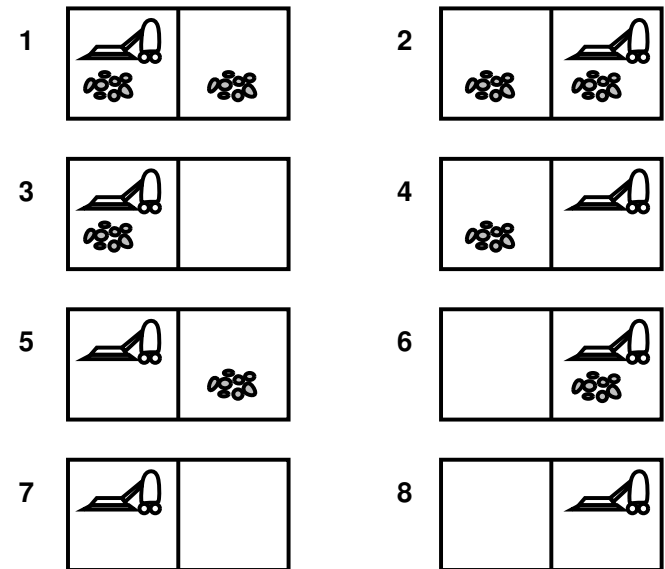
Đa trạng thái, bắt đầu ở $\{1, 2, 3, 4, 5, 6, 7, 8\}$
ví dụ: *Phải* đi đến $\{2, 4, 6, 8\}$. ˘Giải pháp??

Dự phòng, bắt đầu ở #5

Định luật Murphy: *Hót* có thể làm bẩn một
tấm thảm đang sạch

Cảm biến cục bộ: chỉ biết có bụi và vị trí
hiện tại.

˘Giải pháp??



Khởi tạo bài toán trạng thái đơn

Một *bài toán* được định nghĩa bởi bốn mục:

◦ *trạng thái ban đầu* ví dụ: “ở Arad”

◦ *toán tử* (hoặc *hàm kế tiếp* $S(x)$)

ví dụ: Arad $\rightarrow$ Zerind Arad $\rightarrow$ Sibiu v.v.

◦ *kiểm tra mục tiêu*, có thể là

tường minh, ví dụ: $x =$ “ở Bucharest”

ngầm định, ví dụ: $NoDirt(x)$

◦ *chi phí đường đi* (cộng dồn)

ví dụ: tổng khoảng cách, số lượng toán tử đã thực thi, v.v.

Một *giải pháp* là một chuỗi các toán tử
dẫn từ trạng thái ban đầu đến một trạng thái mục tiêu

Chọn lựa một không gian trạng thái

Thế giới thực thì phức tạp một cách vô lý

⇒ không gian trạng thái phải được *trừu tượng hóa* để giải quyết bài toán

(Trừu tượng) trạng thái = tập hợp các trạng thái thực

(Trừu tượng) toán tử = kết hợp phức tạp của các hành động thực

ví dụ: “Arad $\rightarrow$ Zerind” đại diện cho một tập hợp phức tạp

các tuyến đường có thể, đường vòng, trạm dừng chân, v.v.

Để đảm bảo tính khả thi, bất kỳ trạng thái thực nào “ở Arad”

đều phải đi được đến *một* trạng thái thực nào đó “ở Zerind”

(Trừu tượng) giải pháp =

tập hợp các đường đi thực tế là giải pháp trong thế giới thực

Mỗi hành động trừu tượng phải “dễ dàng” hơn bài toán ban đầu!

Ví dụ: Câu đố 8 ô (8-puzzle)

5	4	
6	1	8
7	3	2

Start State

1	2	3
8		4
7	6	5

Goal State

- ⌞ trạng thái??
- ⌞ toán tử??
- ⌞ kiểm tra mục tiêu??
- ⌞ chi phí đường đi??

Ví dụ: Câu đố 8 ô (8-puzzle)

5	4	
6	1	8
7	3	2

Start State

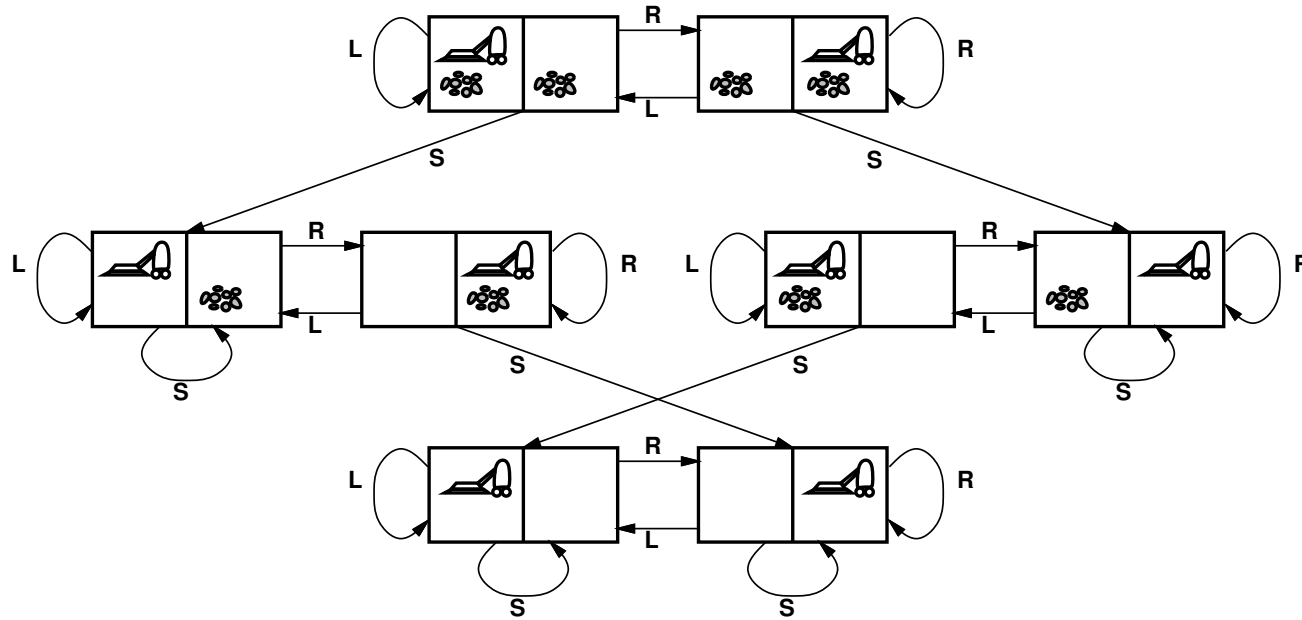
1	2	3
8		4
7	6	5

Goal State

- trạng thái??: vị trí nguyên của các ô vuông (bỏ qua các vị trí trung gian)
- toán tử??: di chuyển ô trống sang trái, phải, lên, xuống (bỏ qua kẹt v.v.)
- kiểm tra mục tiêu??: = trạng thái mục tiêu (được cho trước)
- chi phí đường đi??: 1 cho mỗi lần di chuyển

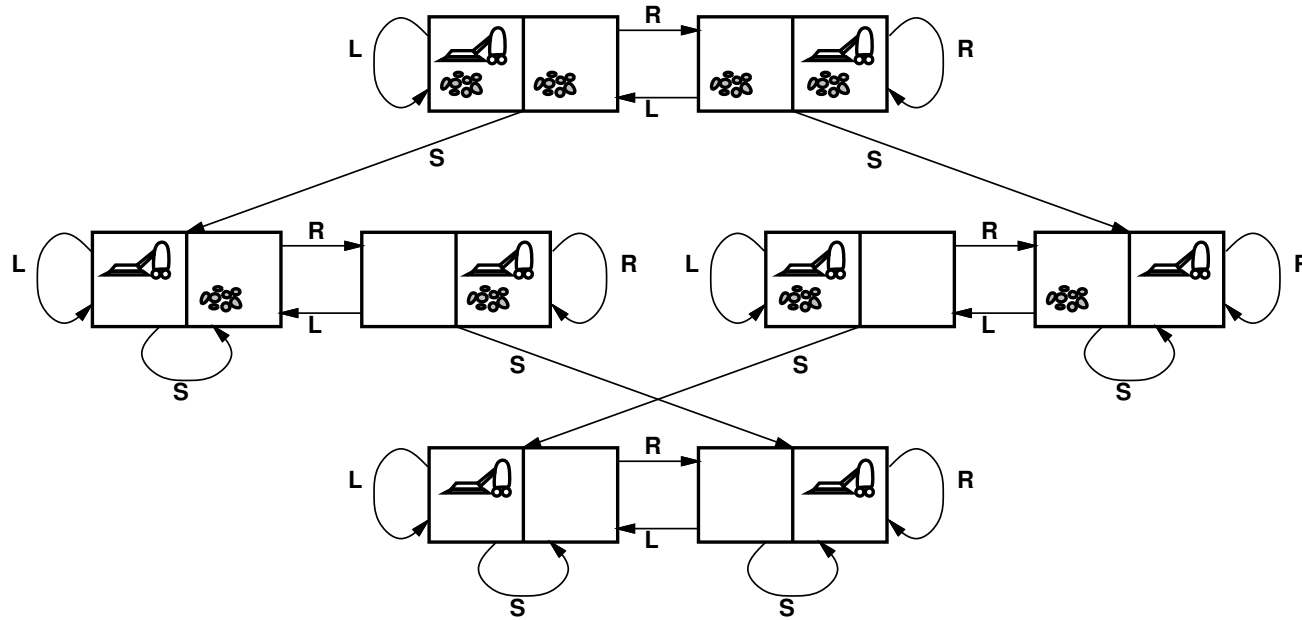
[Lưu ý: giải pháp tối ưu của họ Câu đố n -ô là NP-khó]

Ví dụ: đồ thị không gian trạng thái thế giới máy hút bụi



- ◌ trạng thái??
- ◌ toán tử??
- ◌ kiểm tra mục tiêu??
- ◌ chi phí đường đi??

Ví dụ: đồ thị không gian trạng thái thế giới máy hút bụi



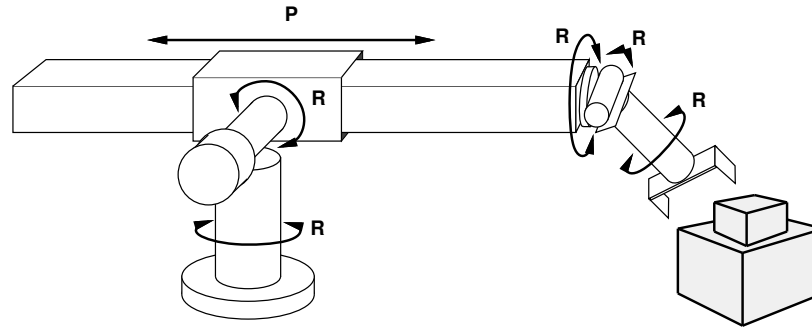
◡trạng thái??: số nguyên biểu thị vị trí bụi và robot (bỏ qua *lượng* bụi)

◡toán tử??: *Trại, Phãi, Hót*

◡kiểm tra mục tiêu??: không còn bụi

◡chi phí đường đi??: 1 cho mỗi toán tử

Ví dụ: lắp ráp robot



⌞trạng thái??: tọa độ giá trị thực của

các khớp robot

các bộ phận của vật thể cần lắp ráp

⌞toán tử??: chuyển động liên tục của các khớp robot

⌞kiểm tra mục tiêu??: lắp ráp hoàn chỉnh *không tính robot!*

Chi phí đường đi?: thời gian thực thi

Các thuật toán tìm kiếm

Ý tưởng cơ bản:

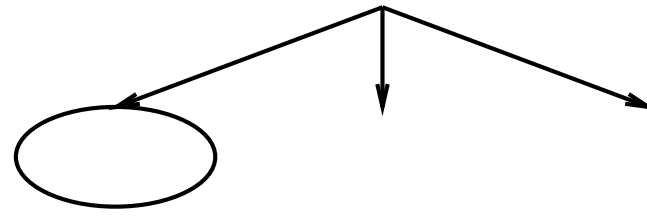
khám phá ngoại tuyến, mô phỏng không gian trạng thái

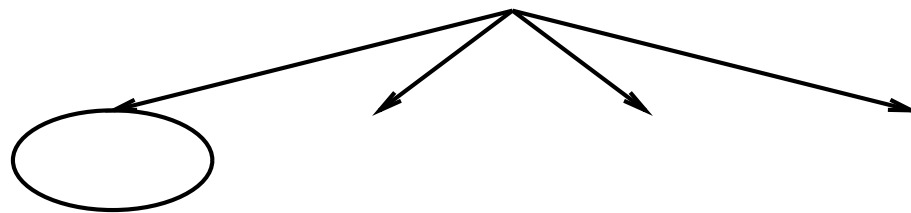
bằng cách sinh ra các trạng thái kế tiếp từ những trạng thái đã khám phá

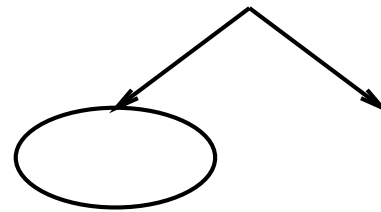
(còn gọi là *khai triển* trạng thái)

```
function GENERAL-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

Ví dụ tìm kiếm tổng quát







Cài đặt các thuật toán tìm kiếm

```
function GENERAL-SEARCH(problem, QUEUING-FN) returns a solution, or failure
nodes ← MAKE-QUEUE(MAKE-NODE(INITIAL-STATE[problem]))
loop do
  if nodes is empty then return failure
  node ← REMOVE-FRONT(nodes)
  if GOAL-TEST[problem] applied to STATE(node) succeeds then return node
  nodes ← QUEUING-FN(nodes, EXPAND(node, OPERATORS[problem]))
end
```


Cài đặt (tiếp theo): trạng thái (states) vs. nút (nodes)

Một *trạng thái* là một (biểu diễn của) cấu hình vật lý

Một *nút* là một cấu trúc dữ liệu tạo thành một phần của cây tìm kiếm

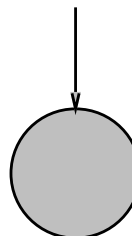
bao gồm *nút cha*, *các nút con*, *độ sâu*, *chi phí đường đi $g(x)$*

Trạng thái không có nút cha, nút con, độ sâu, hay chi phí đường đi!

State

5	4	
6	1	8
7	3	2

Node



Hàm **EXPAND** (Khai triển) tạo ra các nút mới, điền vào các trường khác nhau và sử dụng **OPERATORS** (Toán tử) (hoặc **SUCCESSORFN**) của bài

toán để tạo ra các trạng thái tương ứng.

Các chiến lược tìm kiếm

Một chiến lược được xác định bằng cách chọn *thứ tự khai triển nút*

Các chiến lược được đánh giá dựa trên các khía cạnh sau:

✓ tính hoàn chỉnh (completeness)—nó có luôn tìm ra giải pháp nếu giải pháp tồn tại không?

✓ độ phức tạp thời gian—số lượng nút được tạo/khai triển

✓ độ phức tạp không gian—số lượng nút tối đa lưu trong bộ nhớ

✓ tính tối ưu (optimality)—nó có luôn tìm ra giải pháp có chi phí thấp nhất không?

Độ phức tạp thời gian và không gian được đo lường thông qua

b —hệ số rẽ nhánh (branching factor) tối đa của cây tìm kiếm

d —độ sâu của giải pháp có chi phí thấp nhất

m —độ sâu tối đa của không gian trạng thái (có thể là ∞)

Các chiến lược tìm kiếm mù (Uninformed search)

Chiến lược *tìm kiếm mù* chỉ sử dụng những thông tin có sẵn trong định nghĩa của bài toán

Tìm kiếm theo chiều rộng (Breadth-first search)

Tìm kiếm chi phí đồng nhất (Uniform-cost search)

Tìm kiếm theo chiều sâu (Depth-first search)

Tìm kiếm sâu hạn chế (Depth-limited search)

Tìm kiếm sâu lặp sâu dần (Iterative deepening search)

Tìm kiếm theo chiều rộng

Khai triển nút chưa khai triển ở độ sâu nông nhất

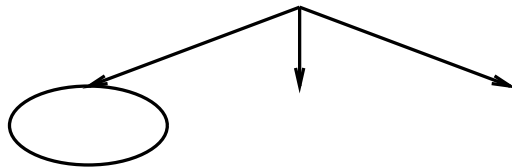
Cài đặt:

QUEUEINGFN (Hàm hàng đợi) = đặt các nút kế tiếp vào cuối hàng đợi

•

•

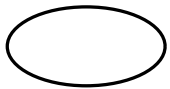
•



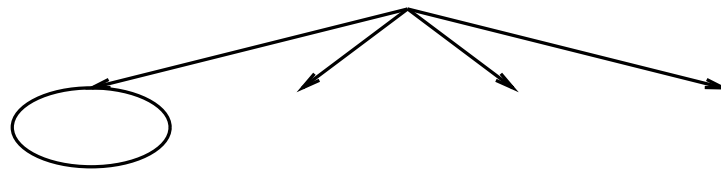
•

•

•



-
-
-



Các thuộc tính của tìm kiếm theo chiều rộng

◡ Hoàn chỉnh??

◡ Thời gian??

◡ Không gian??

◡ Tối ưu??

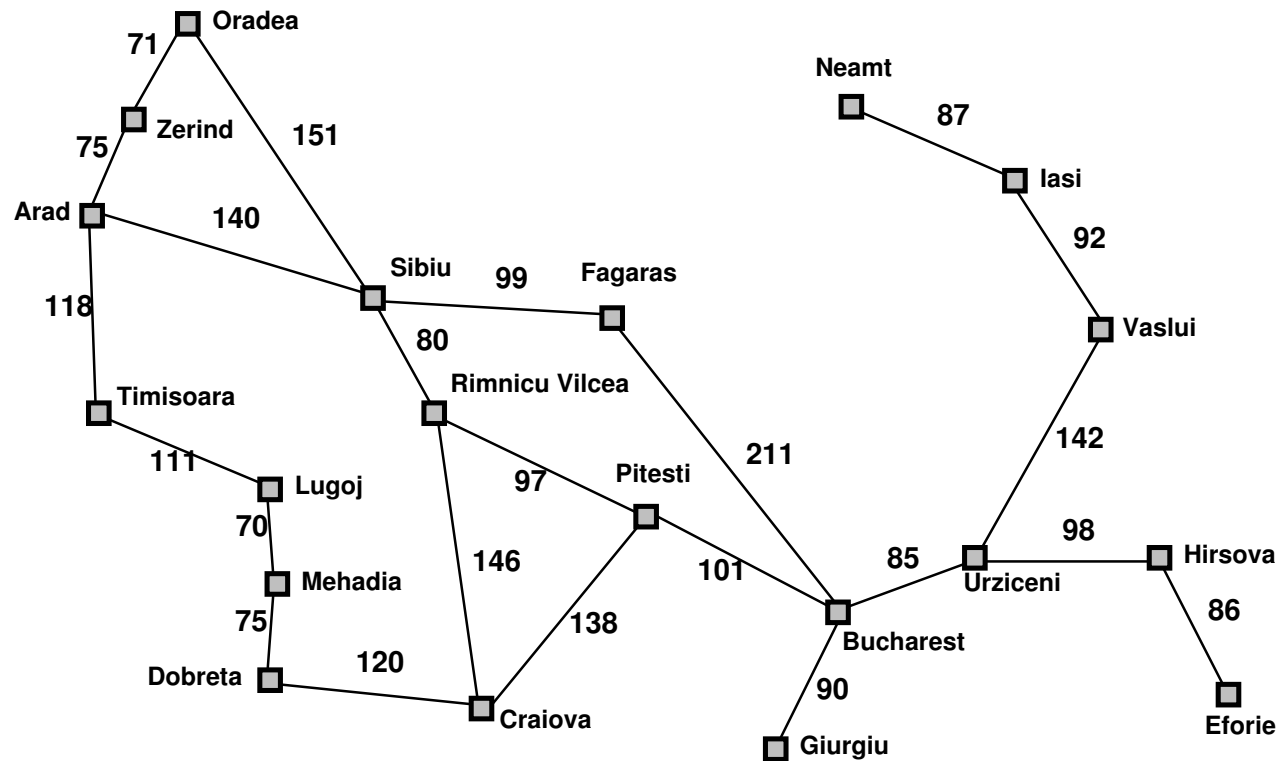
Các thuộc tính của tìm kiếm theo chiều rộng

- ◡ Hoàn chỉnh?? Có (nếu b là hữu hạn)
- ◡ Thời gian?? $1 + b + b^2 + b^3 + \dots + b^d = O(b^d)$, tức là tăng theo hàm mũ của d
- ◡ Không gian?? $O(b^d)$ (lưu giữ mọi nút trong bộ nhớ)
- ◡ Tối ưu?? Có (nếu chi phí = 1 cho mỗi bước); thông thường thì không tối ưu

Không gian là một vấn đề lớn; có thể dễ dàng tạo ra các nút ở mức 1MB/giây

do đó 24 giờ = 86GB.

Romania với chi phí bước tính bằng km



Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

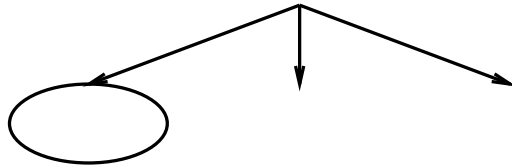
Tìm kiếm chi phí đồng nhất

Khai triển nút chưa khai triển có chi phí thấp nhất

Cài đặt:

$\text{QUEUEINGFN} = \text{chèn theo thứ tự chi phí đường đi tăng dần}$

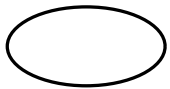
-
-
-



•

•

•



-

-

-



Các thuộc tính của tìm kiếm chi phí đồng nhất

- ◡ Hoàn chỉnh?? Có, nếu chi phí mỗi bước $\geq \epsilon$
- ◡ Thời gian?? Số nút có $g \leq$ chi phí của giải pháp tối ưu
- ◡ Không gian?? Số nút có $g \leq$ chi phí của giải pháp tối ưu
- ◡ Tối ưu?? Có

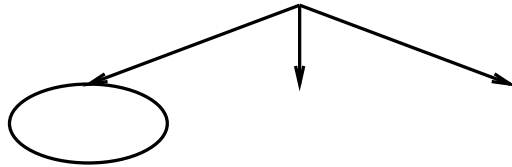
Tìm kiếm theo chiều sâu

Khai triển nút chưa khai triển ở độ sâu sâu nhất

Cài đặt:

$QUEUEINGFN =$ chèn các nút kế tiếp vào đầu hàng đợi

-
-
-



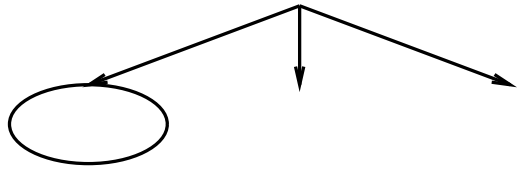
-

-

-



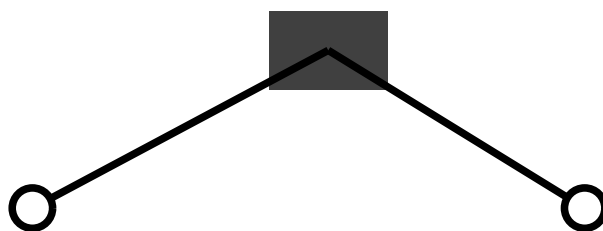
-
-
-

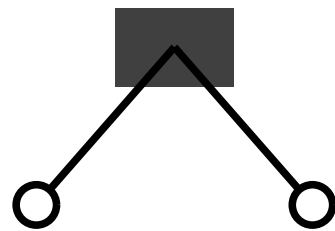


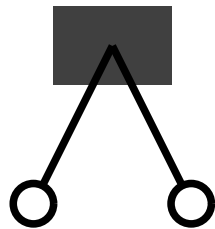
Tức là, tìm kiếm theo chiều sâu có thể rơi vào các vòng lặp vô hạn
Cần một không gian tìm kiếm hữu hạn, không có chu trình (hoặc phải kiểm tra trạng thái lặp lại)

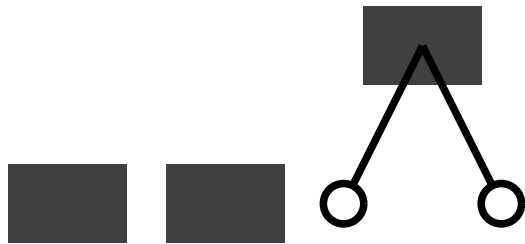
Tìm kiếm theo chiều sâu trên cây nhị phân độ sâu 3



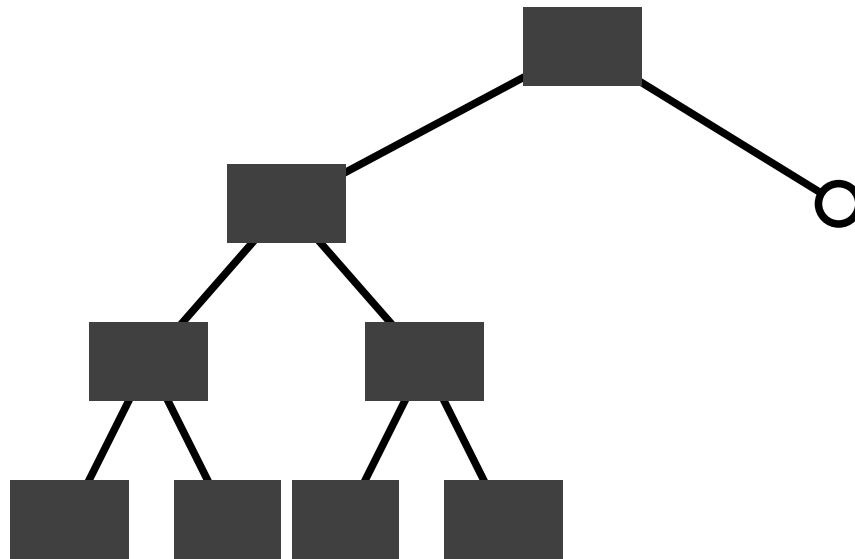


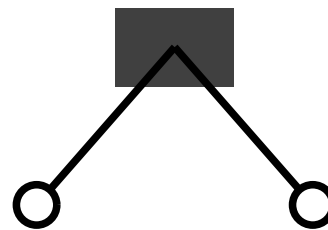


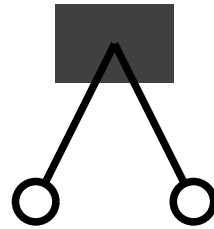


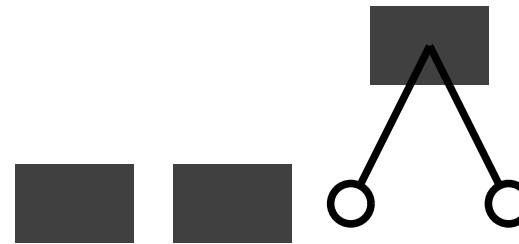


Tìm kiếm theo chiều sâu trên cây nhị phân độ sâu 3 (tiếp)









Các thuộc tính của tìm kiếm theo chiều sâu

◡ Hoàn chỉnh??

◡ Thời gian??

◡ Không gian??

◡ Tối ưu??

Các thuộc tính của tìm kiếm theo chiều sâu

⌞ Hoàn chỉnh?? Không: thất bại trong không gian có độ sâu vô hạn, không gian có chu trình

Sửa đổi để tránh lặp lại các trạng thái trên đường đi

⇒ hoàn chỉnh trong các không gian hữu hạn

⌞ Thời gian?? $O(b^m)$: rất tệ nếu m lớn hơn nhiều so với d

nhưng nếu các giải pháp dày đặc, có thể nhanh hơn nhiều so với tìm kiếm theo chiều rộng

⌞ Không gian?? $O(bm)$, tức là không gian tuyến tính!

⌞ Tối ưu?? Không

Tìm kiếm sâu hạn chế (Depth-limited search)

= tìm kiếm theo chiều sâu với giới hạn độ sâu là l

Cài đặt:

Các nút ở độ sâu l không có nút con kế tiếp

Tìm kiếm sâu lặp sâu dần (Iterative deepening search)

function ITERATIVE-DEEPENING-SEARCH(*problem*) **returns** a solution sequence

inputs: *problem*, a problem

for *depth* $\leftarrow$ 0 **to** ∞ **do**

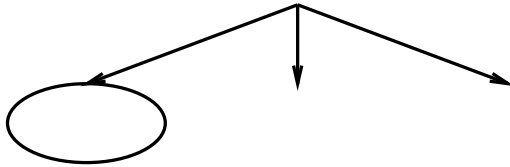
result $\leftarrow$ DEPTH-LIMITED-SEARCH(*problem*, *depth*)

if *result* $\neq$ cutoff **then return** *result*

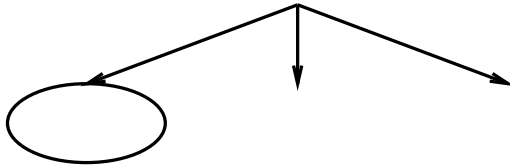
end

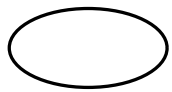
Tìm kiếm sâu lặp sâu dần $l = 0$

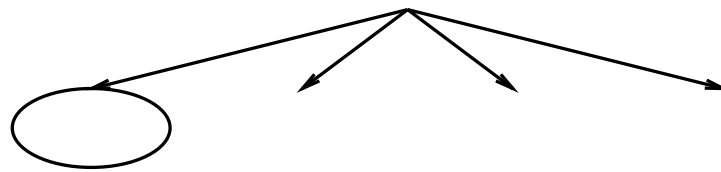
Tìm kiếm sâu lặp sâu dần $l = 1$



Tìm kiếm sâu lặp sâu dần $l = 2$









Các thuộc tính của tìm kiếm sâu lặp sâu dần

◦ Hoàn chỉnh??

◦ Thời gian??

◦ Không gian??

◦ Tối ưu??

Các thuộc tính của tìm kiếm sâu lặp sâu dần

✓ Hoàn chỉnh?? Có

✓ Thời gian?? $(d + 1)b^0 + db^1 + (d - 1)b^2 + \dots + b^d = O(b^d)$

✓ Không gian?? $O(bd)$

✓ Tối ưu?? Có, nếu chi phí bước = 1

Có thể sửa đổi để khám phá trên cây chi phí đồng nhất

Tóm tắt

Khởi tạo bài toán thường yêu cầu trừu tượng hóa các chi tiết của thế giới thực để định nghĩa một không gian trạng thái có thể khả thi để khám phá

Nhiều chiến lược tìm kiếm mù khác nhau

Tìm kiếm sâu lặp sâu dần chỉ sử dụng không gian tuyến tính và không tốn nhiều thời gian hơn các thuật toán tìm kiếm mù khác